Learning to Generate Handwritten Digits

FVSBN, neural autoregression and adversarial learning

CS5494 Topics in Generative AI | Assignment 1
Huang ChuanBin | Student ID: 60434764
Individual submission

1. Introduction

I implement three unconditional generative models on MNIST. First, a fully visible sigmoid belief network (FVSBN) models every binary pixel with logistic regression on preceding pixels. Second, a masked neural autoregressive model (MADE) adds nonlinear hidden layers while preserving the same probability factorization. Third, a convolutional GAN generates original grayscale images from Gaussian noise. My aim is to compare model capacity, sample quality, training behavior and computational cost, rather than judge the models by a single loss.

In these runs, FVSBN achieves 77.60 nats/image and the two-layer MADE achieves 84.63 on threshold-binarized test images. The GAN is evaluated using uncurated samples, training trajectories and classifier-based diversity diagnostics; its adversarial loss is not a likelihood and cannot be compared numerically with autoregressive NLL.

2. Data and experimental protocol

Item	Choice and reason
Dataset	Original MNIST: 60,000 official training images and 10,000 official test images. All images remain 28 x 28.
Split	A fixed permutation (seed 2026) partitions official training data into 55,000 training and 5,000 validation images. The official test set remains held out.
AR preprocessing	Scale bytes to [0,1], then set x = 1 when intensity >= 0.5 and 0 otherwise. Flatten in row-major raster order. This is fixed threshold binarization, not stochastic binarization.
GAN preprocessing	Use original grayscale values scaled to [-1,1]. No thresholding, resizing or augmentation. GAN trains on the same 55,000-image subset.
Labels	Not supplied to FVSBN, MADE, generator or discriminator. Labels are used only to train an independent diagnostic digit classifier.

All generative runs use seed 42. AR configurations use equal epoch budgets, batch size 256 and the same validation/test split. I select each AR checkpoint by minimum validation NLL and report its test NLL afterward. GAN evaluation uses the predefined final epoch. These choices separate model selection from held-out evaluation.

3. Autoregressive methods

3.1 Joint distribution and objective

Following the chain-rule formulation in Lectures 2 and 3 [1,2], I order the D = 784 binary pixels from top left to bottom right. Each conditional is Bernoulli with probability q_i:

$$p(\mathbf{x}) = \prod_{i=1}^{D} p(x_i \mid x_{<i}), \quad p(x_i \mid x_{<i}) = q_i^{x_i}(1-q_i)^{1-x_i}$$

$$\mathrm{NLL}(\mathbf{x}) = -\sum_{i=1}^{D} [x_i \log q_i + (1-x_i)\log(1-q_i)]$$

I minimize the average image NLL with Adam, using binary_cross_entropy_with_logits for numerical stability. The loss is summed over pixels, then averaged over images. Bits per pixel = NLL/(784 ln 2). Training evaluates all conditionals in one masked forward pass using observed prefixes; generation must instead sample pixels sequentially from previously generated prefixes.

3.2 FVSBN: linear conditional logits

$$q_i = \mathrm{sigmoid}\left(b_i + \sum_{j<i} W_{ij} x_j\right)$$

A strictly lower-triangular mask on a 784 x 784 weight matrix enforces j < i. There are 784 x 783 / 2 + 784 = 307,720 active parameters. The implementation stores the dense matrix for efficient batch operations, so it allocates 615,440 parameters; masked entries are inactive. The first pixel has only a bias. This is a generative joint model built from logistic conditionals, rather than a digit classifier. Its limitation is that each conditional logit is linear in the prefix.

3.3 MADE: nonlinear conditional logits

I use MADE [3] as the neural autoregressive model. It follows the nonlinear conditional modeling idea discussed for NADE in Lecture 3, but uses explicit masks for parallel likelihood evaluation. The main network is 784 - 512 - 512 - 784 with ReLU hidden activations and Bernoulli output logits. A one-hidden-layer 784 - 512 - 784 network tests the effect of depth.

Input and output pixel i have degree i. A hidden unit k of width H has degree floor(k(D-1)/H)+1, for k = 0,...,H-1. A hidden connection is retained when the source degree is no greater than the destination degree. The output connection is retained only when the hidden degree is strictly less than the output degree. Every path into output i therefore starts at an input j < i. Masks are fixed registered buffers and are saved in checkpoints.

All AR output biases start at the clamped empirical training pixel log-odds (probabilities clipped to [0.001,0.999]); other weights use PyTorch defaults. This accelerates fitting the mostly black background. During generation, I draw u_i uniformly and set $x_i = 1[u_i < q_i]$. I reuse the same uniforms for progress grids, so changes reflect the model rather than a new sampling seed.

3.4 Correctness checks

Tests verify zero gradients to present/future pixels, exact normalization on all four-pixel states, and agreement between 20,000 samples and enumerated probabilities. FVSBN sampling evaluates only the required weight row and prefix; MADE evaluates the full network at each pixel.

4. GAN method and evaluation

4.1 Architecture

Module	Layers and output shapes
Generator G	z ~ N(0,I), 64 dimensions; Linear 64 -> 6,272; BatchNorm1d + ReLU; reshape to 128 x 7 x 7; transposed convolution 128 -> 64 (kernel 4, stride 2, padding 1) -> 64 x 14 x 14; BatchNorm2d + ReLU; transposed convolution 64 -> 1 with the same settings -> 1 x 28 x 28; Tanh.
Discriminator D	Convolution 1 -> 32 (kernel 4, stride 2, padding 1) -> 32 x 14 x 14; LeakyReLU(0.2); convolution 32 -> 64 with the same settings -> 64 x 7 x 7; LeakyReLU(0.2); flatten; Linear 3,136 -> 1 logit. No output sigmoid in the loss implementation.

The generator has 552,513 parameters and the discriminator has 36,513. The generator creates all pixels in one forward pass. Batch normalization operates in training mode for updates and evaluation mode for sampling. The GAN is unconditional and uses neither an encoder nor digit labels.

4.2 Alternating optimization

I use the adversarial framework [4] with the non-saturating generator objective. Writing d(x) = sigmoid(D(x)), the implemented losses are:

L_D = BCE(D(x), a) + BCE(D(G(z).detach()), 0)
L_G = BCE(D(G(z)), 1)

For the main run, a = 0.9 applies one-sided smoothing to real discriminator labels; fake targets remain 0 and generator targets remain 1. A controlled ablation sets a = 1.0. Each minibatch performs one D update and one G update, with Adam learning rate 0.0002 and betas (0.5,0.999) for both networks. Fake images are detached for the D step; D parameters are frozen for the G step while gradients still flow through D to G. I reuse the generated batch between these two steps. Both runs use the same architecture, seed, data and 30-epoch budget.

4.3 Stability, coverage and novelty diagnostics

I inspect generator/discriminator loss trajectories together with samples from fixed latent vectors. A classifier trained only on labeled training images supplies a diagnostic predicted-digit histogram for 2,000 fresh generated images. The classifier consists of two 3 x 3 convolution/ReLU/max-pool blocks (32 and 64 channels), then 3,136 -> 128 -> 10 linear layers with ReLU. It trains for five epochs using Adam (0.001), selecting the checkpoint by validation accuracy.

For class proportions p_k, I report entropy H = -sum$_k$ p_k ln p_k and exp(H), the effective number of represented classes (maximum 10). I also report classifier confidence and nearest-training-image pixel RMSE for the first 100 generated samples against all 55,000 training images. These are diagnostic proxies: a confident classifier can misread artifacts, balanced class counts can hide within-class collapse, and pixel distance is not a perceptual similarity metric.

There is no claim of a directly comparable GAN likelihood or an ImageNet-based FID score. The generated samples and the conditional likelihood measurements answer different questions.

5. Quantitative results and ablations

Model / change	Train n	LR	Best ep.	Val NLL	Test NLL (SE)	Bits/pixel
FVSBN	55,000	0.001	30	78.72	77.60 (0.20)	0.1428
MADE 1 layer	55,000	0.001	30	80.53	79.26 (0.25)	0.1458
MADE 2 layers	55,000	0.001	30	86.14	84.63 (0.29)	0.1557
MADE low LR	55,000	0.0003	30	94.11	92.41 (0.30)	0.1701
MADE 10k data	10,000	0.001	30	106.50	104.38 (0.34)	0.1921

Table 1. Lower is better. NLL is in nats per 28 x 28 threshold-binarized image. Parentheses are standard errors across 10,000 test images, not uncertainty across training seeds. Every configuration has a 30-epoch budget; the best validation checkpoint may occur earlier.

The two-layer MADE changes test NLL by +7.02 nats/image relative to FVSBN. The paired FVSBN-minus-MADE difference is -7.02 nats/image, with an approximate 95% interval [-7.30, -6.75] over test images for these fixed fitted models. This interval does not establish robustness to random initialization.

Depth, learning rate and data quantity

Depth: adding a second 512-unit layer changes validation NLL from 80.53 to 86.14 (difference +5.62). This is a practical depth comparison, not a parameter-matched ablation: the second layer also adds capacity and computation.

Learning rate: reducing Adam's step size from 0.001 to 0.0003 changes validation NLL from 86.14 to 94.11. The interpretation is limited to this fixed epoch budget; a slower learning rate may require more training. The best validation configuration among the five is FVSBN.

Data: reducing the two-layer model's training set from 55,000 to 10,000 images changes validation NLL from 86.14 to 106.50. The subset is nested and the held-out splits are identical. Equal epochs imply fewer optimizer steps for the smaller dataset, so this measures a practical data-and-compute reduction, not a pure data effect with matched update counts.

Measured compute

Model	Active / stored parameters	Train wall (s)	100 images (s)
FVSBN	307,720 / 615,440	8.3	0.3303
MADE 1 layer	402,704 / 804,112	34.8	0.5388
MADE 2 layers	534,544 / 1,066,768	42.6	1.1006
GAN (0.9)	G 552,513; D 36,513	62.9	0.0008

Table 2. Hardware: NVIDIA GeForce RTX 3060 Laptop GPU; Python 3.12.14, PyTorch 2.5.1+cu121. Wall time excludes setup/data loading but includes epoch validation and progress output. Inference excludes disk I/O and is the median of three GPU-synchronized runs after warm-up, batch size 100. Active AR parameters exclude masked entries. Update-only times and individual timing repeats are in metrics.json.

6. Learning curves and generated samples

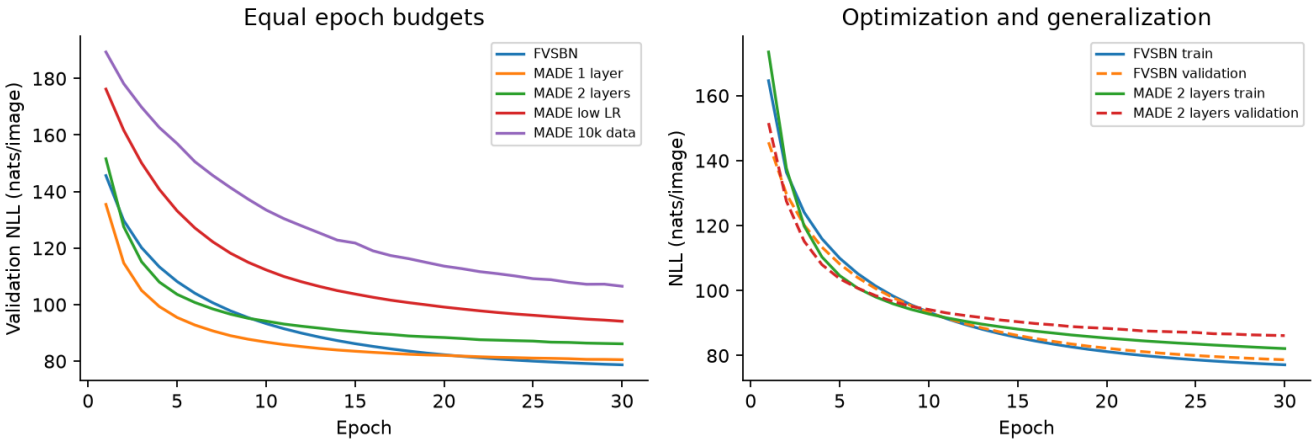


Figure 1. Validation curves for all AR configurations and training/validation curves for the two main AR models. Training NLL is averaged while weights change within an epoch; validation NLL is measured afterward, so their small differences are not a strict generalization-gap estimate.

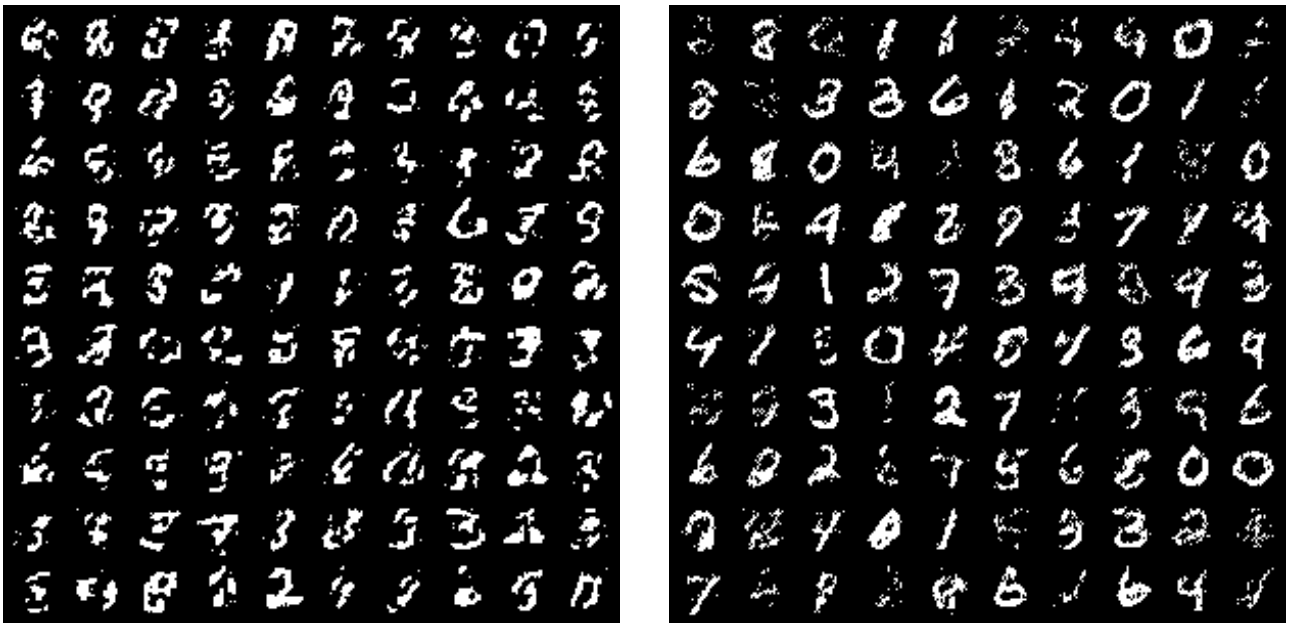


Figure 2. First 100 uncurated samples from FVSBN (left) and two-layer MADE (right). Both use seed 1234 sampling uniforms and their validation-selected checkpoints. White pixels are 1 and black pixels are 0. These are sampled binary pixels, not probability heat maps.

The grids expose both successful and ambiguous digits. The likelihood comparison uses the same binary representation and exact normalized distributions, whereas visual inspection emphasizes recognizable stroke structure. I retain every sample in the fixed grid to avoid overstating quality through selection. Lower likelihood loss alone does not guarantee that every ancestral sample is recognizable.

7. Generation during training

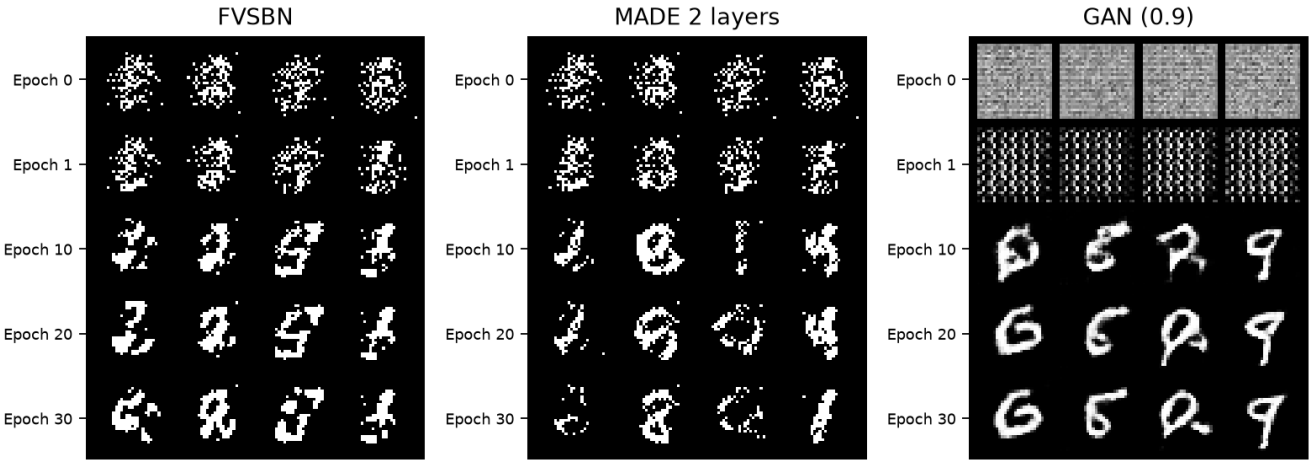


Figure 3. Five stages and four fixed samples per stage for each model. AR models reuse the same uniform random values; GAN reuses the same four latent vectors. Epoch 0 is the untrained initialization (AR biases already match training pixel marginals). Later rows show the actual saved epoch states, which can differ from a validation-selected final AR checkpoint.



Figure 4. First 100 uncurated images from the final main GAN checkpoint. Intensities are mapped from [-1,1] back to [0,1] for display; they are not binarized.

8. GAN stability and diversity

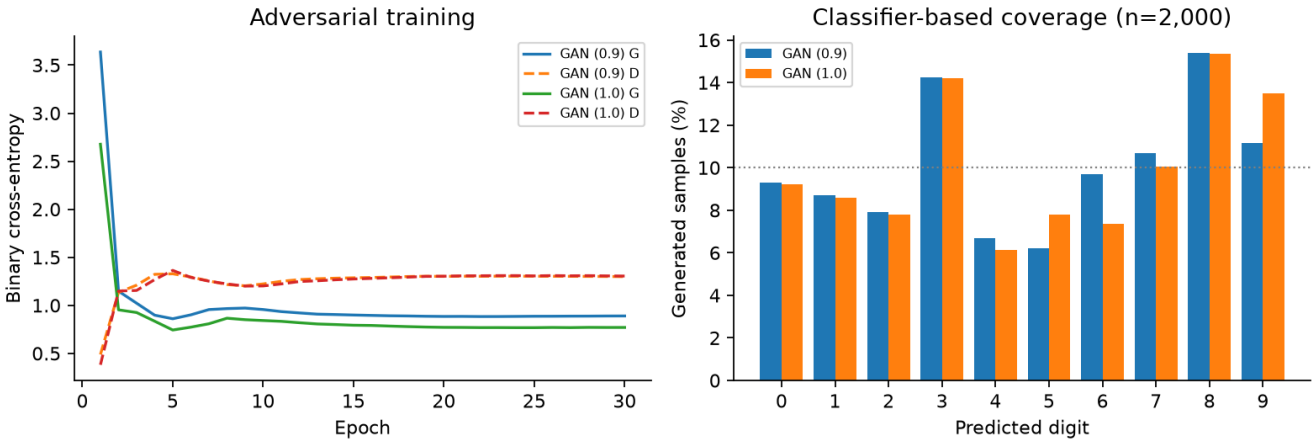


Figure 5. Loss trajectories and predicted-digit frequencies for two equally budgeted GAN runs. The dotted 10% line is a uniform reference, not an enforced class prior. Discriminator losses have different real targets between runs and are therefore not directly comparable measures of quality.

Diagnostic	Real label 0.9	Real label 1.0
Class entropy (nats)	2.263	2.258
Effective classes / 10	9.61	9.56
Mean maximum class probability	0.852	0.858
Fraction with confidence >= 0.9	0.563	0.582
Mean nearest-train pixel RMSE	0.2018	0.2048
Minimum nearest-train pixel RMSE	0.1103	0.0707

Table 3. Coverage/confidence use 2,000 fresh generated images per run (seed 999). Nearest-neighbor diagnostics use the first 100. The auxiliary classifier obtains 98.48% validation and 98.81% official-test accuracy on real grayscale MNIST.

The main GAN receives predictions in 10 of 10 classes, with 9.61 effective classes. This assesses class-level coverage on a finite sample; it cannot exclude repeated writing styles within a digit class. The no-smoothing ablation yields 9.56 effective classes and mean confidence 0.858, compared with 0.852 for smoothing. These observations come from one seed per configuration, so I do not infer that smoothing is universally beneficial.

GAN losses need not fall monotonically because the opponent changes throughout training. Oscillations are therefore interpreted alongside the fixed-latent progress images and diversity histogram. A finite loss without class or visual diversity would not count as successful training. Both ablation sample grids and their complete histories are included in the shared results folder.

I do not observe complete collapse to one digit: the grid contains different digit shapes and all ten predicted classes occur. However, malformed loops, broken strokes and ambiguous digits remain visible. Class-level coverage alone cannot rule out partial or within-class mode collapse.

9. Longer training: a follow-up experiment

The 30-epoch MADE validation curves were still improving near the budget limit. I therefore ran the same two-layer architecture from the same initialization for 120 epochs and also extended the smoothed GAN to 100 epochs. These are follow-up budget experiments, separate from the equal-budget ablations above. AR checkpoint selection still uses validation NLL; the longer GAN uses its predefined final epoch.

Model / budget	Validation NLL	Test NLL	Train wall (s)
MADE / 30 epochs	86.14	84.63	42.6
MADE / 120 epochs	83.57	82.20	64.6
GAN / 30 epochs	Not defined	Not defined	62.9
GAN / 100 epochs	Not defined	Not defined	209.6

Table 4. The longer MADE checkpoint is selected at epoch 111. Its test NLL changes by -2.42 nats/image relative to the 30-epoch model. Its stored architecture and preprocessing are unchanged; additional training does not guarantee a better result than the linear baseline.

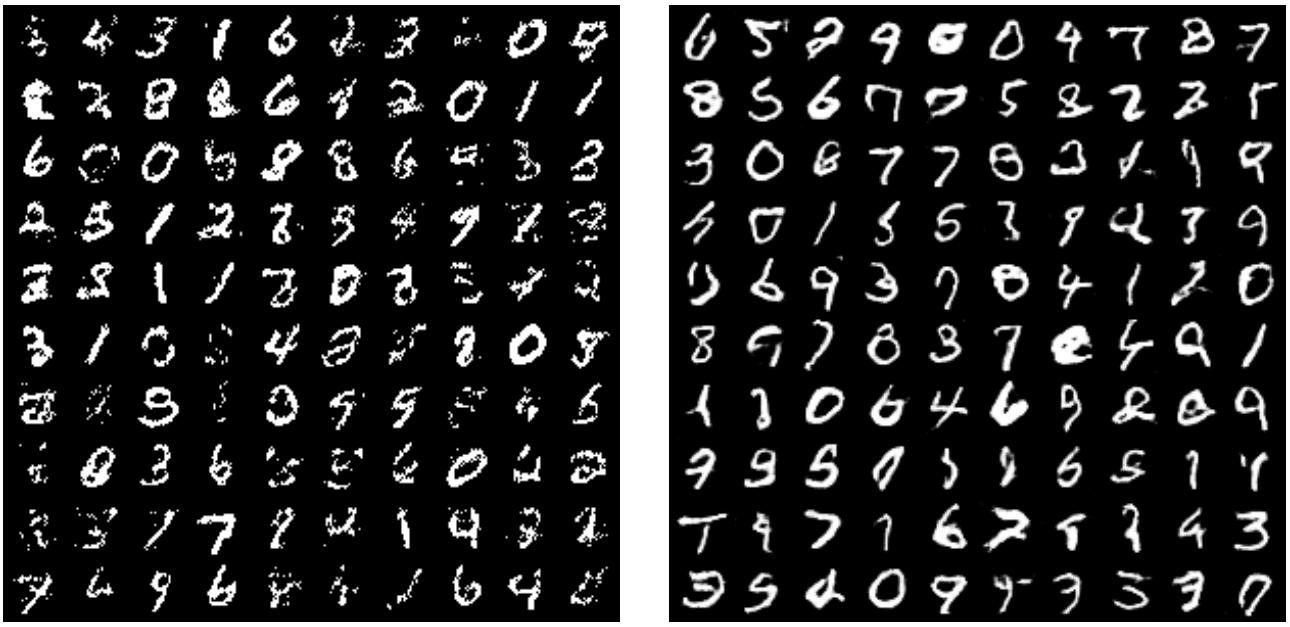
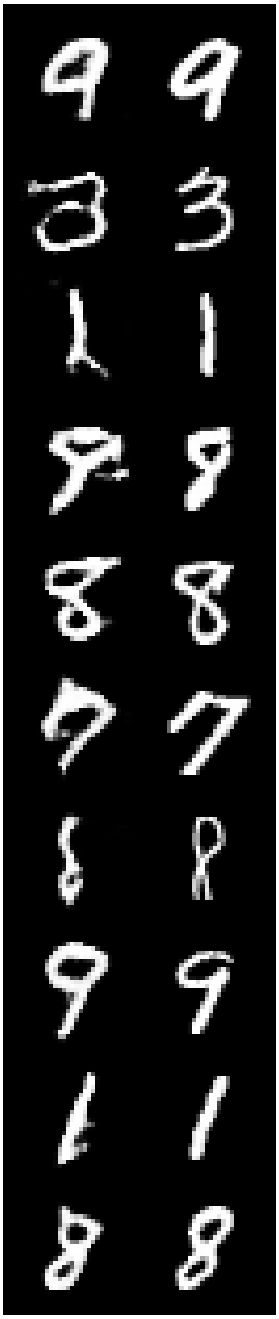


Figure 6. First 100 uncurated outputs after longer training: two-layer MADE (left) and GAN (right), with the same seed-1234 sampling inputs as the shorter runs. Complete progress grids and epoch histories are included in their result folders.

The 100-epoch GAN has 9.76 effective predicted classes, mean classifier confidence 0.887, and confidence >= 0.9 in 67.7% of 2,000 generated images. The 30-epoch values are 9.61, 0.852 and 56.3%. These quantify changes in the diagnostic classifier's response, not a guaranteed improvement in human-perceived realism.

10. Discussion and conclusion

Generated / nearest training



Likelihood and visual quality. FVSBN provides a compact conceptual baseline and exact likelihood, but its linear logits limit conditional interactions. MADE increases nonlinear capacity while preserving a valid autoregressive distribution. A GAN learns a different objective and retains grayscale detail, so I avoid treating its sharper or softer appearance as evidence of better binary density estimation.

Sampling cost. The GAN takes 0.0008 s for 100 images, compared with 1.1006 s for this MADE implementation. The difference is consistent with one-pass GAN sampling versus 784 sequential AR steps. This is an implementation-specific benchmark; caching hidden computations could speed up neural autoregressive sampling.

Memorization check. The first 100 GAN outputs have a minimum nearest-training RMSE of 0.1103. The adjacent pairs show the first ten outputs and their closest training images. This checks for close pixel-space copies in this sample, but does not prove absence of memorization or measure semantic originality.

Limitations. The ablations use one training seed, a limited hyperparameter grid and fixed training budgets. Test-image standard errors capture sampling variability across test images only. Fixed thresholding discards antialiasing and differs from stochastic-binarized MNIST benchmarks, so published NLL figures on other preprocessing should not be compared directly. Classifier metrics may overrate unrealistic digits.

Next experiment. I would repeat configurations across seeds, match optimizer steps in the data-size study, and compare equally sized one- and two-layer MADE models. For GANs, I would inspect per-class variation and add a feature-space two-sample evaluation with a validated MNIST embedding, rather than relying on classifier confidence alone.

Conclusion. These experiments demonstrate the three required generation mechanisms and their main trade-offs. Explicit autoregressive likelihood supports quantitative density comparison; neural conditionals change the capacity and optimization trade-off; adversarial generation produces grayscale images efficiently but requires multiple checks of stability and diversity. Reproducible evidence is more informative than selecting only the best-looking digits.

Figure 7. Each row contains a generated sample from the 30-epoch GAN (left) and its closest image among all 55,000 training images (right), using pixel RMSE. No nearest-neighbor result is used to select GAN checkpoints.

Appendix A. Reproduction and evidence

Code, data, checkpoints and logs: https://github.com/mosthandsomeguy/CS5494-Assignment-1

The accompanying folder contains models.py, train.py, test_models.py, build_report.py, requirements.txt, README.txt, run_colab.ipynb, raw MNIST data, split_indices.npz, and results for every configuration. Each result contains a checkpoint, per-epoch history, measured metrics and sample/progress images. Per-image AR test losses permit paired comparisons. The auxiliary classifier checkpoint and GAN diagnostics are also supplied.

A.1 Google Colab / local commands

1. Select a GPU runtime in Colab and upload/extract the folder, or create a local Python 3.12 environment.
2. Change into the submission folder.
3. Install dependencies and run the checks, experiments and report builder:

```
pip install -r requirements.txt
python test_models.py
python train.py --stage smoke
python train.py --stage all
python build_report.py
```

The notebook run_colab.ipynb provides the same sequence. Stage selectors ar, gan and evaluate allow separate runs. A completed experiment is skipped when its metrics.json exists; use a fresh folder without results for a clean rerun. The data loader verifies/downloads MNIST using torchvision and reconstructs the seeded split. CPU fallback is supported but slower. Do not change budgets while silently reusing previous result folders.

The nine generative runs took 9.0 minutes of measured training wall time in total on the reported GPU, excluding setup, downloads, timing benchmarks and classifier diagnostics. Exact times and outputs vary across devices. The code sets seeds but does not guarantee bitwise-identical CUDA results across platforms.

A.2 Evidence map

Requirement	Evidence
Three tasks / preprocessing / modules	Sections 1-4; models.py and train.py
Generated images and training progress	Figures 2-4 and 6; each results subfolder
Hyperparameters and ablations	Tables 1, 3, 4 and Figures 1, 5
Training and inference runtime	Table 2; metrics.json, including update-only time
Code clarity and runnable instructions	README.txt, notebook, comments and mathematical tests
Discussion / stability / limitations	Sections 8-10 and nearest-neighbor inspection

Before submission, confirm that the shared link is accessible to the marker and includes both code and data. Submit 60434764.pdf on Canvas. Preserve the submitted code/data without subsequent edits, as required by the assignment.

References

[1] Jianyuan Guo. CS5494 Lecture 2: Basic of Probability Distributions. Course slides supplied with the assignment. Relevant material: chain rule, Bayesian networks, logistic regression and neural conditional models.

[2] Jianyuan Guo. CS5494 Lecture 3. Course slides supplied with the assignment. Relevant material: autoregressive factorization, FVSBN (slide 11), neural autoregressive density estimation (slides 13-16).

[3] Mathieu Germain, Karol Gregor, Iain Murray and Hugo Larochelle. MADE: Masked Autoencoder for Distribution Estimation. ICML, 2015. https://proceedings.mlr.press/v37/germain15.html

[4] Ian Goodfellow et al. Generative Adversarial Nets. NeurIPS, 2014. https://arxiv.org/abs/1406.2661

[5] PyTorch contributors. torchvision.datasets.MNIST documentation and dataset downloader. https://docs.pytorch.org/vision/main/generated/torchvision.datasets.MNIST.html

Implementation provenance

The accompanying scripts implement the models with PyTorch layers and optimizers. All reported generative checkpoints are trained from scratch on MNIST; no pretrained generator or external synthetic images are used. The report tables and plots are built from saved run outputs. Course definitions and the cited papers inform the methods; the diagnostic classifier is trained separately and does not supply gradients to any generative model.

Development assistance: AI assistance was used to develop and review code and report text. Numerical results and sample figures are produced by executing the supplied experiments. The student should review the implementation, findings and applicable course disclosure requirements before submission.